

1 论文主体部分笔记

- 背景提示：为了把研究重点放在内存分配和布局变化上，论文首先把可变长数组简化为只在“远端”增长和收缩的结构，其行为类似于栈。论文希望同时控制三类性能，即数组在稳定状态下的存储空间、`grow/shrink` 执行期间的峰值空间，以及 `access`、`grow`、`shrink` 的时间复杂度。旧研究通常把前两种空间合并成一个“任意时刻最大空间”指标；本文最重要的概念变化，是把它们分别记为 $s(N)$ 和 $t(N)$ ，由此得到一条可参数化的空间与更新时间权衡。
- 全文论证结构：第 1–3 节定义问题并介绍旧的 $N + O(\sqrt{N})$ 基线；第 4 节证明稳定冗余和临时冗余不能同时很小；第 5–7 节构造与该空间下界匹配的数据结构；第 8–9 节进一步证明 `grow` 的摊还时间下界；第 10 节说明最优性结论的适用范围和开放问题。
- 研究对象的边界：本文研究的动态数组只允许在末端增加或删除元素，因此是普通数组与栈之间的一种推广。它需要支持创建空数组 `Array()`、返回长度 `Length()`、按下标读取 `Get(i)`、按下标修改 `Set(i,a)`、末端加入 `Grow(a)` 和末端删除 `Shrink()`，其中 `access` 同时包含 `Get` 与 `Set` 所需的元素定位能力。双端版本可以用两个单端动态数组组合得到；任意位置插入或删除还需要维护后续元素的秩和下标，属于更困难的动态序列问题，不是本文的研究对象。
- 动态数组需要内存管理的原因：普通数组存放在一段连续内存中，而数组末端之后的地址可能已经被其他对象占用，因此动态数组无法保证原地延长，只能重新申请更大的连续区域并复制元素，或者把元素分散到多个可动态管理的数据块中。
- 论文采用的内存模型：论文中的实现由动态申请的固定长度块（blocks）组成。数据块（data block）保存数组元素，索引块（index block）保存指向其他块的指针，辅助块（auxiliary block）保存长度和计数等信息；除主索引块（main index block）外，每个可访问块都必须由某个指针标识。论文把所有已分配块的长度之和定义为空间使用量，因此未使用的块内位置、索引指针和辅助数组都会计入额外空间。
- 内存模型中的时间和不可压缩性假设：论文为了简化分析，假设申请或释放任意固定长度块需要 $O(1)$ 时间；即使申请长度为 L 的块需要 $O(L)$ 时间，主要摊还结论仍可以通过相应记账维持。元素被视为任意的 w 位字符串，一般不能把 N 个元素无损压缩到少于 N 个机器字；每个独立数据块还需要一个占 $\Theta(1)$ 个机器字的地址指针。这些是理论模型假设，而不是对真实内存分配器的直接描述。
- 最朴素的精确容量方案：每次 `grow` 或 `shrink` 都申请一个容量恰好等于新长度的数组，复制全部元素后释放旧数组。它的稳定空间可以达到 $N + O(1)$ ，但每次扩缩容需要 $\Theta(N)$ 时间，并在复制过程中短暂保存新旧两个数组，峰值空间约为 $2N + O(1)$ 。这个例子说明稳定表示紧凑，并不代表更新时间和临时空间也小。
- 普通几何扩缩容方案：容量满时申请约 $1 + \alpha$ 倍的新数组，并在使用率下降到更低阈值时

缩容。它支持最坏 $O(1)$ 的 `access` 和摊还 $O(1)$ 的 `grow/shrink`，但稳定冗余与重建峰值空间都是 $\Theta(N)$ 。 α 越小，稳定浪费越少；但两次扩容之间只有约 αN 次 `grow`，而一次扩容需要复制约 N 个元素，因此每次 `grow` 分摊的复制成本约为 $1/\alpha$ 。缩容还必须设置滞后区间，否则 `grow` 和 `shrink` 在边界附近交替时会频繁重建。

- 双空间指标的定义与意义：若一个结构在操作完成后最多使用 $N + s(N)$ 空间，并在一次 `grow` 或 `shrink` 执行过程中最多使用 $N + t(N)$ 空间，则称为一个 $(s(N), t(N))$ -实现，其中 s 和 t 为非递减函数。 $s(N)$ 表示长期稳定冗余， $t(N)$ 表示扩缩容时允许达到的临时冗余。区分二者的实际动机是：如果一个应用同时保存许多动态数组，而同一时刻只调整其中少数数组，那么大多数数组可以一直保持紧凑，系统只需为正在调整大小的数组提供临时工作空间。
- Sitarski 的 HAT 结构：HAT 使用一个长度约为 B 的索引块和约 $\lceil N/B \rceil$ 个长度为 B 的数据块，并保持 $B = \Theta(\sqrt{N})$ ，通常令 B 为 2 的幂。元素 i 位于第 $\lfloor i/B \rfloor$ 个数据块的第 $i \bmod B$ 个位置，因此可以在最坏 $O(1)$ 时间定位。额外空间来自索引块、最后一个未填满数据块以及可能保留的空块，合计为 $O(B) = O(\sqrt{N})$ ；当数组规模越过阈值时改变 B 并重建，两次重建之间的大量更新可以摊还重建成本。HAT 的名称虽然来自 hashed array tree，但结构中实际没有哈希，其形态更接近索引块指向数据块列表。
- Brodnik 等人的结构：其基本版本使用大小为 $1, 2, 3, \dots$ 的递增数据块。前 k 个块的总容量为 $\Theta(k^2)$ ，所以容纳 N 个元素只需要 $\Theta(\sqrt{N})$ 个块和同量级指针，最后一个部分填充块造成的浪费也只有 $O(\sqrt{N})$ 。基本版本从下标反推出块号时涉及三角数和平方根；另一种变体把数据块组织成大小为 2 的幂的虚拟超级块，以便使用移位等字操作定位。与 HAT 相比，Brodnik 结构通常不需要在参数变化时周期性移动全部数据元素，但定位计算更复杂；二者都达到 $N + O(\sqrt{N})$ 这一目标。
- Theorem 4.1 的结论与证明设置：定理说明，即使操作序列中只有 `grow` 和 `access`，任何动态数组实现也必定在某些时刻使用 $N + \Omega(\sqrt{N})$ 空间。证明先从空数组开始连续执行 N 次 `grow`，并观察第 N 次 `grow` 完成后的稳定状态。此时全部 N 个元素分布在 k 个连续数据块中，这些数据块的总容量至少为 N 。证明接下来不分析具体算法，而只根据 k 的大小分成“块很多”和“块很少”两种情况，因此能够覆盖论文模型中的任意实现。
- Theorem 4.1 的证明过程：若 $k \geq \sqrt{N}$ ，那么即使所有数据块都完全填满，结构仍然必须保存至少 k 个指向这些块的指针，否则某些块无法被 `access` 找到。元素本身至少需要 N 个机器字，块指针又需要至少 k 个机器字，因此

$$\text{space} \geq N + k \geq N + \sqrt{N}.$$

在这种情况下，下界直接出现在执行完 N 次 `grow` 后的稳定状态中，其来源不是块内空位，而是大量块带来的索引开销。

若 $k < \sqrt{N}$ ，由于 k 个数据块必须共同容纳 N 个元素，由平均值原理，至少存在一个容量为 ℓ 的块满足

$$\ell \geq \frac{N}{k} > \sqrt{N}.$$

设这个大块是在数组长度为某个 $N' \leq N$ 时申请的。就在申请完成而元素尚未复制进去的时刻，新块仍为空，原有的 N' 个元素必须完整保存在其他旧块中，所以当时总空间至少为 $N' + \ell$ 。又因为

$$\ell > \sqrt{N} \geq \sqrt{N'},$$

所以该历史时刻的空间严格大于 $N' + \sqrt{N'}$ 。这里的下界不一定出现在最终状态，而可能出现在过去某次 `grow` 分配大块时的瞬时峰值。

- Theorem 4.1 的核心含义：证明揭示了动态数组分块表示中的不可兼得关系，即块很多会产生大量指针开销，块很少则必然存在大块，而大块刚申请时必须与旧表示短暂共存。 $\sqrt{N}$ 是平衡“块数量”和“最大块容量”时自然出现的尺度。定理只保证操作历史中的某些时刻出现 $N + \Omega(\sqrt{N})$ ，不能表述成数组在所有时刻都浪费 $\sqrt{N}$ 空间，也不能把下界理解为 `grow` 的时间下界。

- Theorem 4.2 的结论与推广思路：Theorem 4.1 给出的是稳定冗余和临时冗余处于同一 $\sqrt{N}$ 尺度时的对称下界，Theorem 4.2 则把这种关系推广为任意 $(s(N), t(N))$ -实现都必须满足

$$s(N)t(N) \geq N.$$

其思路仍然是“块数与最大块不能同时很小”，但不再把二者预先平衡在 $\sqrt{N}$ ，而是直接用稳定空间上限 $s(N)$ 限制块数，再由此推出临时空间下限。

- Theorem 4.2 的证明过程：从空数组开始执行 N 次 `grow`，并观察最终稳定状态。由于每个数据块至少需要一个指针，而稳定额外空间最多为 $s(N)$ ，保存元素的数据块数量至多为 $s(N)$ ；即使稳定空间中还需要索引表空位和辅助信息，这些开销也只会进一步减少能够使用的块数。现在 N 个元素分布在至多 $s(N)$ 个块中，所以至少有一个数据块 B 的容量满足

$$|B| \geq \frac{N}{s(N)}.$$

设这个块是在数组长度为 $N' \leq N$ 时申请的，刚申请时它还没有承载旧元素，原有元素仍必须保存在其他块中，因此该时刻至少出现 $N/s(N)$ 的临时额外空间，即

$$t(N') \geq \frac{N}{s(N)}.$$

因为定义要求 t 非递减，且 $N' \leq N$ ，所以

$$t(N) \geq t(N') \geq \frac{N}{s(N)}.$$

两边乘以 $s(N)$ ，便得到 $s(N)t(N) \geq N$ 。

- Theorem 4.2 中需要注意的证明条件： $t(N)$ 的单调性用于把较早规模 N' 时出现的峰值转换成关于最终规模 N 的下界；如果没有这一条件，只能得到关于 $t(N')$ 的结论。新块之所以全部算作临时冗余，是因为它刚申请时尚未替代旧表示，旧元素不能提前消失。证明只使用 `grow` 和 `access` 已经足够，因为下界在更弱的操作集合中成立后，也自然适用于额外允许 `shrink` 的完整结构。若允许压缩任意元素或不使用常规指针表示块，则需要重新检查证明的内存模型假设。

- Corollary 4.3 与指数关系：将 $s(N) = O(N^{1/r})$ 代入 Theorem 4.2，可以得到

$$t(N) = \Omega\left(\frac{N}{N^{1/r}}\right) = \Omega\left(N^{1-1/r}\right).$$

因此 $r = 3$ 时自然出现 $N^{1/3}$ 与 $N^{2/3}$ ， $r = 4$ 时自然出现 $N^{1/4}$ 与 $N^{3/4}$ ，两个指数之和为 1，并不是任意选择。第 5 节构造 $r = 3$ 的匹配实现，第 6 节推广到一般 r ，第 7 节再在给定范围内去掉稳定空间中的额外因子 r ，从而使空间上界与第 4 节的乘积下界闭合。

- 空间下界与时间下界的区别：Theorem 4.1 和 Theorem 4.2 只讨论空间，并没有证明 **grow** 的 $\Omega(r)$ 时间下界；后者来自第 8–9 节的增长博弈分析，而且只对论文定义的标准实现（standard implementations）严格成立。因此空间最优性与时间最优性是两条不同的证明线：第 4 节给出空间下界，第 5–7 节给出匹配构造，第 8–9 节再补上更新时间下界，三者结合后才构成论文标题所指的较完整最优性。
- 理论结果的实践边界：论文没有实验比较缓存局部性、分配器元数据、内存对齐、碎片化和实现常数。其“optimal”指给定 word-RAM 与分块内存模型下的渐近时空权衡最优，并不等价于在实际程序中一定优于 **vector**、**ArrayList** 等几何扩容结构。

2 关键问题 Q&A

Q1：为什么论文的大部分内容假设可变长数组只在“远端”增长和收缩？它如何扩展到更一般的情况？

A1：论文要研究的是数组长度改变时，如何在保持随机访问的同时减少内存浪费。如果允许在任意位置插入或删除，还必须动态维护后续元素的秩和下标，这会变成另一个更困难的问题。论文的结果可以扩展到两端增长和收缩：分别使用两个单端可变长数组保存序列的左右两部分，即可得到双端可变长数组。需要注意，这种“双端扩展”不等于支持任意位置插删；后者的复杂度明显更高。

Q2：论文所研究的可变长数组需要支持哪些基本操作？

A2：它支持创建空数组、返回长度、按下标读取 **Get(i)**、按下标修改 **Set(i,a)**、在末端加入元素 **Grow(a)**，以及删除末端元素 **Shrink()**。论文始终要求 **Get** 和 **Set** 保持最坏 $O(1)$ 时间，并研究 **grow**、**shrink** 的时间与空间成本。

Q3：为什么普通的倍增动态数组仍然存在值得研究的问题？

A3：倍增法能实现最坏 $O(1)$ 访问和摊还 $O(1)$ 扩缩容，但会长期保留大量尚未使用的容量，总空间只能写成 $O(N)$ ，而不是更接近理论最低值 N 。减小增长因子可以降低浪费比例，却会增加复制频率和摊还更新时间，因此传统方法本身已经隐含了一个空间与更新时间之间的权衡。

Q4：能否只用 $N + o(N)$ 空间保存大小为 N 的动态数组，同时仍保持常数时间操作？

A4：可以。Sitarski 的 HAT 和 Brodnik 等人的结构已经实现了 $N + O(\sqrt{N})$ 空间，并保持常

数时间访问和摊还常数时间更新。Brodnik 等人还证明，在把所有时刻使用的空间统一计算的传统模型下，某些时刻出现 $N + \Omega(\sqrt{N})$ 空间是不可避免的。

Q5: 既然已有 $N + \Omega(\sqrt{N})$ 下界，本文为什么还能得到小于 $\sqrt{N}$ 的稳定冗余？

A5: 本文没有推翻旧下界，而是区分了两种空间：数组完成操作后长期保存所需的空间，以及 `grow` 或 `shrink` 执行期间短暂需要的峰值空间。旧下界说明某些时刻必须使用较多空间，但这个时刻可以只发生在重组过程中；数组在其他时间仍然可以具有更紧凑的表示。

Q6: 如何形式化“稳定存储空间”和“扩缩容临时空间”？

A6: 论文定义 $(s(N), t(N))$ -实现：稳定保存大小为 N 的数组时最多使用 $N + s(N)$ 空间，执行 `grow` 或 `shrink` 时最多使用 $N + t(N)$ 空间。 $s(N)$ 衡量长期冗余， $t(N)$ 衡量操作期间的临时冗余。论文的主要目标就是刻画二者之间可以达到的最优关系。

Q7: 稳定冗余 $s(N)$ 和临时冗余 $t(N)$ 能否同时很小？

A7: 不能。定理 4.2 证明任何这类实现都必须满足 $s(N)t(N) \geq N$ 。直观上，如果稳定状态只允许很少的块指针，就只能使用较少、较大的数据块；当某个大块刚被申请而旧数据尚未释放时，必然出现较大的临时空间。于是若 $s(N) = O(N^{1/r})$ ，就必须有 $t(N) = \Omega(N^{1-1/r})$ 。

Q8: 为什么论文首先选择 $(O(N^{1/3}), O(N^{2/3}))$ 作为特殊例子？

A8: 这是第一个既突破 $N + O(\sqrt{N})$ 稳定空间、结构又足够简单的例子。令 $B = \Theta(N^{1/3})$ ，使用大小为 B 的小块和大小为 B^2 的大块。大块始终填满，小块保存数组末端并吸收日常增删，因此稳定冗余只有 $O(B) = O(N^{1/3})$ ；合并 B 个小块时必须申请一个大小为 $B^2 = N^{2/3}$ 的新块，所以临时冗余为 $O(N^{2/3})$ 。这与 Q7 的下界匹配。

Q9: 第 5 节的两级结构如何推广到任意 $r \geq 2$ ？

A9: 取 $B = \Theta(N^{1/r})$ ，建立大小依次为 $B, B^2, \dots, B^{r-1}$ 的 $r-1$ 个数据层级。 B 个 B^i 块可以合并成一个 B^{i+1} 块；反过来，一个高层块也可以逐级拆成低层块。这个过程相当于一个以块数量为数字的 B 进制计数器。

Q10: 为什么一般构造允许每层最多拥有 $2B$ 个块，而不是普通 B 进制中的 B 个？

A10: 这是一个冗余 B 进制表示。当某层达到 $2B$ 个块时，只合并其中 B 个并保留另外 B 个，因此进位后仍有缓冲；缩小时也不必立刻反向拆分。 B 到 $2B$ 之间的余量避免了 `grow` 和 `shrink` 在边界附近交替发生时不断合并、拆分。

Q11: 参数 B 为什么需要随着 N 重建，而不能永久固定？

A11: 空间分析要求 $B = \Theta(N^{1/r})$ 。当增长到 $N = B^r$ 时，论文把 B 加倍并重建；当缩小到 $N = (B/4)^r$ 时，把 B 减半并重建。两个阈值之间留有较宽间隔，因此两次全局重建之间必然经过大量 `grow` 或 `shrink`，重建的 $O(N)$ 成本可以被摊还。

Q12: 为什么一般结构的稳定额外空间是 $O(rN^{1/r})$ ，临时额外空间是 $O(N^{1-1/r})$ ？

A12: 共有 $r-1$ 层，每层最多保存 $2B$ 个块指针，所以索引空间为 $O(rB) = O(rN^{1/r})$ ；最

小层中部分填充块造成的浪费也不超过同一数量级。最大数据块的大小为 $B^{r-1} = N^{1-1/r}$ ，合并、拆分时新旧块短暂共存，因此临时额外空间由这个最大块支配。

Q13: 多层分块以后，为什么仍能按下标在最坏 $O(1)$ 时间访问元素？

A13: 逻辑下标首先被映射为“所属层级、该层中的块编号、块内偏移”。论文令 B 为 2 的幂，并把各层块数量紧凑编码在少数机器字中，再利用 **msb**、**lsb**、移位和位掩码直接确定层级。确定层级后，除以 B^k 和模 B^k 都可用移位和按位与完成。因此理论 word-RAM 模型中只需常数个字操作。实践中因为 r 通常很小，线性检查 $O(r)$ 或二分查找 $O(\log r)$ 反而可能更简单。

Q14: 为什么 **grow** 和 **shrink** 的摊还代价是 $O(r)$ ？

A14: 新元素先进入第 1 层，之后每次被合并并只会上升一层；在一次参数重建周期中，一个元素至多经过 $r-1$ 个层级，因此其搬移次数为 $O(r)$ 。拆分成本则由两次同级拆分之间发生的大量 **shrink** 操作支付。论文用信用法形式化这一点：元素上移会释放信用支付复制，层级积累的信用支付向下拆分和全局重建。

Q15: 能否把第 6 节稳定空间中的额外因子 r 去掉？

A15: 可以在论文给定的参数范围内做到。第 7 节把过大的新块切成若干大小至多为 $t(N)$ 的块，以增加少量指针为代价限制单次分配规模。对

$$r \leq \frac{1}{2} \frac{\log N}{\log \log N},$$

该变换把稳定额外空间从 $O(rN^{1/r})$ 降到 $O(N^{1/r})$ ，同时保持临时空间 $O(N^{1-1/r})$ 、最坏 $O(1)$ 访问和摊还 $O(r)$ 扩缩容。

Q16: $O(r)$ 的摊还更新时间是否只是作者构造造成的，还是不可避免的？

A16: 对论文定义的广泛标准实现，它是不可避免的。第 9 节证明：如果稳定空间只有 $N + O(rN^{1/r})$ ，且 $r = O(\log N)$ ，那么 **grow** 的摊还代价至少为 $\Omega(r)$ 。因此第 6 节的 $O(r)$ 上界与下界匹配；第 7 节去掉稳定空间中的因子 r 后，也没有消除更新时间中的 r 。

Q17: 什么是增长博弈，论文为什么需要它？

A17: 增长博弈把动态数组的具体指针布局抽象成“向有限个子数组逐个加入元素，并在必要时合并已有子数组”的单人博弈。一次合并的成本等于被重新复制的元素数。精确求解这个博弈后，可以证明在块数量和空闲位置受限时，反复合并具有不可避免的总代价，从而把数据结构的更新时间问题转化为一个可证明的组合优化下界。

Q18: 为什么一般情况下不能把 **grow** 和 **shrink** 的摊还界变成相同的最坏情况界？

A18: 常见的去摊还方法是在后台逐步建立新结构，但这会让旧结构和新结构在一段较长时间内同时存在，从而长期占用本应只在瞬间允许的临时空间，破坏 $N + O(N^{1/r})$ 的稳定空间目标。论文因此指出，除 $r = 2$ 的特殊情形外，所给出的 $O(r)$ 更新时间不能在保持相同空间权衡的同时直接改成最坏情况界。

Q19: 论文的最优性结论覆盖所有可能的动态数组实现吗？

A19: 空间乘积下界具有较广的适用性，但 $\Omega(r)$ 时间下界只严格证明到了“标准实现”：新块分配后，旧块中的元素按序复制进去并随即释放旧块。作者相信非标准编码不会带来渐近优势，但若要覆盖所有算法，还需要对元素和指针作适当的不可压缩性假设。这是论文最优性结论的重要边界。

Q20: 论文最后留下了哪些开放问题？

A20: 主要有两个。第一，能否把 $\Omega(r)$ 的摊还时间下界从标准实现推广到所有实现，并给出严格的不可压缩性论证；第二，增长博弈虽然被精确求解，但证明较长，是否存在更简单的归纳或其他方法，只证明足以推出渐近最优下界的估计，而不必完整刻画所有最优状态与最优走法。

Q21: 这套理论结果在实践中应当如何评价？

A21: 它最适合“长期同时保存许多动态数组，但任一时刻只调整少数数组”的场景，因为大多数数组可以保持极紧凑，正在调整的数组短暂借用较多空间。不过论文主要给出渐近理论结果，没有通过实验比较缓存局部性、分配器开销、常数因子和实现复杂度。因此“optimal”指理论模型中的时空权衡最优，并不等价于在真实系统中一定快于倍增数组。